

프로그래밍 언어 간 의미 거리의 정량적 측정 방법

(Abstract) 프로그래밍 언어 사이의 의미 거리는 대개 직관에 의존하고 있으나, 이 논문은 언어 사이의 의미 거리를 정량적으로 측정하는 방법을 제안한다. 의미 거리 측정이 어려운 이유는 의미론의 복잡성 때문이기도 하지만, 정량적 측정 방법의 부재가 주요 원인이다. 이 논문은 왕복 번역(RTT: round-trip translation)을 이용한 언어 거리 측정 방법을 제안하고, 상이한 패러다임의 언어 사이의 의미 거리를 측정한다. 객체지향 언어인 C++를 중심으로 하여 언어 거리를 측정한 결과, Python과의 의미 거리는 6.89, Java와는 7.17인 것에 비해, 함수형 프로그래밍 언어인 Haskell과의 의미 거리는 36.83, 논리형 프로그래밍 언어인 Prolog와의 의미 거리는 225.00인 것으로 나타났다. 이러한 프로그래밍 언어 사이의 정량적 의미 거리는 다중 언어 개발 환경에서 프로그램 마이그레이션 등에 실질적으로 활용될 수 있을 것으로 기대된다.

1. 서론

이 논문은 프로그래밍 언어 사이의 의미 거리를 정량적으로 측정하는 방법을 제시한다. 프로그래밍 언어 자체의 크기를 정량적으로 측정하려는 시도는 구문론적 관점에서는 일부 시도된 바 있으나 의미론적 관점에서는 행해진 바 없다. 그 이유는 프로그래밍 언어 의미론의 복잡성 때문이기도 하지만, 더 근본적인 원인은 의미론 측면에서 정량적 측정 방법을 고안하기 어렵기 때문이다. 이 논문은 프로그래밍 언어 간 의미 거리를 정량적으로 측정하는 방법을 제시함으로써 다중 언어 개발, 프로그램 마이그레이션 등의 이론적 토대를 마련하고자 한다.

제안한 방법은 언어 사이의 트랜스파일(transpile) 기법을 바탕으로 한다. 고급 언어를 저급 언어로 변환하는 컴파일(compile)에 비해 고급 언어를 다른 고급 언어로 변환하는 트랜스파일은 양쪽 언어의 의미론을 파악해야 하므로 수행하기 어렵다. 이 논문에서는 대규모 언어 모델(LLM)을 활용한 트랜스파일 기법을 활용한다. LLM을 활용한 소스 코드 변환은 개발 생산성을 높이는 핵심 기술로 자리 잡고 있다[1]. 그러나 서로 다른 프로그래밍 언어 사이의 구조적 차이와 계산 모델의 차이가 번역 품질 및 기능 보존에 미치는 영향을 정량적으로 분석한 연구는 아직 충분하지 않다[2].

서로 다른 계산 모델을 따르는 언어 간 번역은 단순히 구문(syntax)을 치환하는 작업이 아니다. 상태 관리, 제어 흐름, 메모리 모델 등 문제 해결 방식을 대상 언어의 의미론에 맞게 재구성해야 하기 때문이다. 따라서 트랜스파일 과정은 계산 모델 간 변환 난이도를 객관적으로 평가할 수 있는 지표가 될 수 있다.

본 연구에서는 C++ 코드를 다른 언어로 트랜스파일한 뒤 다시 C++ 코드로 복원하는 왕복 트랜스파일(RTT: round-trip transpiling)을 수행한다. 이상적인 상황에서는 RTT를 반복적으로 수행하면 고정점(fixed point)을 찾을 수 있는데,

여기서 고정점이란 프로그램의 변화를 더 이상 관찰 수 없는 수렴 상태를 뜻한다[3]. 본 논문에서는 변환된 프로그램 간 유사도가 임계치 이상인 상태를 근사 고정점으로 정의하고, 이에 도달하는 과정을 추적 및 분석하였다.

프로그래밍 언어 사이의 의미 거리 측정이 실질적으로 가능함을 보이기 위해서 본 연구는 C++를 기준 언어로 하여 프로그래밍 언어 간 RTT를 측정하였다. C++는 대표적인 객체지향 언어로서 절차형 프로그래밍 패러다임을 바탕으로 하고 있다. 이와 유사한 언어로 분류되는 Java와 Python에 더하여, 함수형 프로그래밍 언어인 Haskell, 논리형 프로그래밍 언어인 Prolog를 선정하여 프로그래밍 언어 간 의미 거리를 측정한다. 이를 통해 프로그래밍 언어 간 의미 거리가 전통적인 프로그래밍 패러다임의 차이를 정량적으로 측정할 수 있는지 비교 분석한다.

2. 연구 방법론 및 실험 설계

의미 거리 측정에 RTT를 활용하고 있지만, RTT가 항상 성공적으로 이루어지는 것은 아니다. 때론 의미 보존이 이루어지지 않는 경우도 있는데, RTT 사이의 의미 보존과 형태적 안정성을 평가하기 위해 근사 고정점 도달률(P_s)을 측정한다. P_s 는 전체 유효 시도 중 기능 실패 없이 설정된 왕복 한도 내에서 근사 고정점에 도달한 시도의 비율이다.

의미 거리(D)를 측정하기 위해서는, 추가로 두 가지 지표가 필요한데, 이들은 홉 수(number of hops)와 코드 유사도(code similarity)이다. 먼저, 평균 홉 수($\bar{H}_n$)는 근사 고정점 도달에 성공한 n 회 시도의 평균 편도 번역 횟수이다. H_n 은 n 번째 성공 시도의 홉 수인데, 편도 번역 1회를 1홉으로 센다. 유사도(Sim)는 인접한 왕복 단계에서 복원된 코드 간의 유사도인데, 이 논문에서는 JPlag를 활용하여 측정하였다[4]. 본 연구에서는 연속된 두 복원 코드 간의 유사도가 특정 임계치 $\theta = 0.85$ 이상일 때 근사 고정점에 도달한 것으로 판정한다.

의미 거리(D)는 평균 홑 수를 근사 고정점 도달률로 보정한 값($D = \overline{H}_n / P_s$)으로 정의한다. 성공한 시도의 번역 횟수와 근사 고정점에 도달하지 못한 시도의 비율을 함께 반영한 지수로서, 평균 홑 수에 비례하며 도달률에 반비례하는 지표이다.

3. 실험 결과 및 해석

실험 대상 프로그램으로는 LeetCode에서 추출한 15개의 대표적인 알고리즘 문제의 정답 프로그램을 사용하였다. 신뢰성을 위해 실험을 독립적으로 5회 반복하였다. 실험에는 GLM-5.3-Flash를 사용하였다. 이 모델은 총 3200억 개의 매개변수를 가지며, 100만 토큰의 문맥을 지원한다. 코드 생성과 에이전트 작업에서 상위권 성능을 보인다. 생성 파라미터는 temperature = 0.6, top_p = 0.95, max_tokens = 16384로 설정하였으며, 코드 변환의 일관성을 위해 추론 토큰 생성을 억제하도록 설정하였다.

Python과 Java 경유 경로의 근사 고정점 도달률은 각각 74.7%, 82.7%, 거리는 6.89, 7.17으로 측정되었다. Haskell과 Prolog의 도달률은 각각 17.3%와 2.7%로 더 낮았으며, 거리는 각각 36.83과 225.00으로 더 컸다.

유사도의 중앙값은 Python과 Java 경유 경로에서 0.62~0.64로, 인접 왕복 단계의 복원 코드 간 구조적 유사성이 상대적으로 높았다. Haskell과 Prolog 경유 경로의 유사도 중앙값은 각각 0.24와 0.16으로 더 낮아, 인접 왕복 단계 간 코드 형태의 변화가 더 컸음을 보여준다. 이는 RTT 실패 건수에서도 확인할 수 있는데, Python의 경우 13건, Java의 경우 19건임에 비해 Haskell과 Prolog는 각각 62건과 73건으로 상당히 커짐을 볼 수 있다.

근사 고정점 도달에 성공한 시도의 평균 홑 수는 Python과 Java 경유 경로(5.36~5.69)가 Haskell과 Prolog 경유 경로(6.00~6.38)보다 적었다. Haskell과 Prolog에서는 대부분의 시도가 왕복 한도에 도달하기 전에 기능 실패로 중단되었다.

표 1. 계산 모델에 따른 왕복 번역(RTT) 및 거리 결과

경로	성공/유효	P_s	$\overline{H}_n$	Sym	D
Python	62/75	0.827	5.694	0.64	6.89
Java	56/75	0.747	5.357	0.62	7.17
Haskell	13/75	0.173	6.385	0.24	36.83
Prolog	2/75	0.027	6.000	0.16	225.00

4. 논의

절차형 언어에 비해 함수형 언어는 불변성과 순수 함수의 합성을, 논리형 언어는 사실과 규칙의 동형화(unification) 및 되추적을 중심으로 계산을

수행한다. 이러한 계산 모델의 차이로 인해 동일한 알고리즘 문제라도 코드의 문제 해결 구조를 재구성해야 하는데, 실험 결과는 이러한 계산 모델 차이가 의미 거리에 반영되었음을 시사한다.

문제의 특성이 의미 거리에 영향을 주는 경우도 발견할 수 있었다. 특히 재귀적 문제의 경우 이런 현상이 발견되었는데, 하노이탑 문제에서는 함수형 언어와 논리형 언어를 경유하는 경로에서도 초기 2회 왕복까지 기능이 보존되었다. 이는 재귀 구조가 명확하고 상태 변경이 제한된 알고리즘의 경우, 계산 모델 간 변환에서도 초기 단계에는 기능이 보존될 수 있음을 보여준다.

또한, 계산 모델이 서로 다른 언어 간에 LLM을 활용하여 코드를 변환할 때는 반복적인 왕복 번역 과정에서의 오류 누적 양상을 함께 고려해야 한다. 극단적으로 근사 고정점 도달률이 0인 경로의 거리는 사실상 무한대이므로 산출 불가로 처리하였다. Prolog의 거리는 성공 2건에 기반하므로, 정밀한 크기나 배수보다 관찰된 경향을 중심으로 해석한다.

5. 결론

본 논문은 RTT를 활용해 프로그래밍 언어 사이의 의미 거리를 측정하는 방법을 제안하고 실험으로 효과를 검증했다. 평균 홑 수와 근사 고정점 도달률을 바탕으로 거리를 계산하고, 번역 안정성과 기능 보존을 정량적으로 평가했다. 실험 결과, LLM 코드 번역의 성능은 프로그래밍 패러다임에 따라 달랐다. 함수형 및 논리형 언어를 거치는 경로에서는 기능 실패가 증가하고, 근사 고정점 도달률이 낮아졌으며, 보정 거리는 커졌다. 이는 다중 언어 개발과 소프트웨어 마이그레이션에서 계산 모델 간 호환성을 고려해야 함을 시사한다.

참고문헌

- [1] Solovyeva, Lola. et al. Leveraging LLMs for Automated Translation of Legacy Code: A Case Study on PL/SQL to Java Transformation. In Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE '25), 809–813. (2025)
- [2] Chowdhary, K. R. On the Evolution of Programming Languages. arXiv: 2007.02699 (2020)
- [3] 조찬우 외 2인. "왕복 번역과 고정점을 이용한 언어 거리 계산", 2025 한국소프트웨어종합학술대회 논문집, pp.1467–1469. 2025.12
- [4] Prechelt, Lutz. et al. Finding Plagiarisms among a Set of Programs with JPlag. *Journal of Universal Computer Science* 8, 1016–1038. (2002)